

Fundamentals of program development

Strings, lists, tuples

Chalmers



Agenda

The invisible ink

Encoding of characters and strings

Objects and attributes

Mutable objects

Invisible ink

Invisible ink



What would you do if you got an email with the above content?

Maybe you should try to use the invisible ink?

Invisible ink

*The nuclear codes are in the
black suitcase on the bench.
Password kanelbulle.*

01010100011010000110010100100000011011100111010
10110001101101100011001010110000101110010001000
00011000110110111101100100011001010111001100100
00001100001011100100110010100100000011010010110
11100010000001110100011010000110010100100000011
00010011011000110000101100011011010110010000001
11001101110101011010010111010001100011011000010
11100110110010100100000011011110110111000100000
01110100011010000110010100100000011000100110010
10110111001100011011010000010111000100000010100
0001100001011100110111001101110110110111101110
01001100100001000000110101101100001011011100110
01010110110001100010011101010110110001101100011
0010100101110

We have encrypted the text to left with invisible ink. The result can be “seen” to the right.
And you can always decode the invisible message.

String literals: value and output

```
>>> " \t      \t\t\t \t\t \t\t "  
' \t      \t\t\t \t\t \t\t '  
  
>>> print(" \t      \t\t\t \t\t \t\t ")  
  
>>> print("hello\tworld")  
hello  world  
  
>>> print("hello\nworld")  
hello  
world
```

The string literal (in double quotes) is an expression for a string.

When we evaluate a string expression, we see a string literal which is still in the Python syntax. There `\t` encodes the tab symbol.

But when we print, we see the tabs as "long spaces".

Another invisible symbol is `'\n'` (**newline**), which leads to breaking a line into two.

Some more: `'\r'`, `'\b'`, `'\7'`.

Binary encoding of characters

*The nuclear codes are in the
black suitcase on the bench.
Password kanelbulle.*

```
01010100011010000110010100100000011011100111010
10110001101101100011001010110000101110010001000
00011000110110111101100100011001010111001100100
00001100001011100100110010100100000011010010110
11100010000001110100011010000110010100100000011
00010011011000110000101100011011010110010000001
11001101110101011010010111010001100011011000010
11100110110010100100000011011110110111000100000
01110100011010000110010100100000011000100110010
10110111001100011011010000010111000100000010100
0001100001011100110111001101110110110111101110
01001100100001000000110101101100001011011100110
01010110110001100010011101010110110001101100011
```

0010100101110

Just like with numbers, each character in the original text is encoded by a binary number. The binary numbers can also be encoded as a sequence of spaces and tabs:

- 0 = space ' '
- 1 = tab '\t'

Invisible ink

The nuclear codes are in the
black suitcase on the bench.
Password kanelbulle.

[illegible]

The whole message is encrypted with “invisible” symbols:

- space: ' '
- tab: '\t'

Encoding of Characters

ASCII

ASCII = American Standard Code for Information Interchange

Characters are encoded as numbers in the range 0..127. Each is 7 bits long:

'A' → 65 → 1000001 (binary digit == bit)

In this way you can encode 128 characters - not just letters, but also digits, punctuation marks, control codes, and space-like symbols:

'a' → 97 → 1100001

'2' → 50 → 0110010

' ' → 32 → 0100000

'\t' → 9 → 0001001

Note that the encoding for '2' as a character is different from 2 as an integer!

Functions for ASCII and Unicode

```
>>> ord('A')
65
>>> ord('2')
50
>>> ord('ABC')
TypeError: ord() expected a character, but
string of length 3 found

>>> chr(66)
'B'
>>>> chr(32001)
'綬'
>>> ord('绿')
32511
```

ord() shows the character's ASCII code

ord() functions only for strings of length 1

chr() shows the character for a given ASCII-code

chr() works for codes bigger than 127

and ord() can handle non-ASCII characters

Unicode is an extension of ASCII.

ASCII characters have the same codepoints in Unicode, but Unicode can also represent non-Latin languages.

UTF-8 is one of the encodings for Unicode

ASCII table

```
>>> for code in range(256):  
...     print(code,chr(code))  
0  
1  
2  
  
33 !  
34 "  
35 #  
  
65 A  
66 B  
  
252 ü  
253 ý  
254 þ  
255 ÿ
```

A simple for-loop can show all codes in any given range.

First comes a number of control codes.

One of them is “printed” as a beep sound, which one?

From 33 start the punctuation symbols

From 65 are the capital letters without accent, followed by lowercase letters.

Scandinavian and other letters with accents are not included in **ASCII**, which often causes problems in programs which don’t support unicode.

A bit better ASCII table

```
def simpleAsciiTable():
    for a in range(0,32):
        row = ""
        for k in range(0,8):
            row = row + str(32*k+a)
                + ' ' + chr(32*k+a) + '\t'
        print(row)
```

The code

- prints out 32 columns which start with 32*k
- every line has 8 columns

There are control characters in the ranges 0..31 and 130..160

```
>>> simpleAsciiTable()
0      32      64 @      96 `      128 □      160      192 À      224 à
1      33 !      65 A      97 a      129 □      161 ¡      193 Á      225 á
2      34 "      66 B      98 b      130 □      162 ¢      194 Â      226 â
3      35 #      67 C      99 c      131 □      163 £      195 Ã      227 ã
4      36 $      68 D      100 d     132 □      164 ¤      196 Ä      228 ä
5      37 %      69 E      101 e     133      165 ¥      197 Å      229 å
6      38 &      70 F      102 f     134 □      166 ¦      198 Æ      230 æ
7      39 '      71 G      103 g     135 □      167 §      199 Ç      231 ç
8      40 (      72 H      104 h     136 □      168 ¨      200 È      232 è
9      41 )      73 I      105 i     137 □      169 ©      201 É      233 é
10     42 *      74 J      106 j     138 □      170 ª      202 Ê      234 ê
11     43 +      75 K      107 k     139 □      171 «      203 Ë      235 ë
12     44 ,      76 L      108 l     140 □      172 ¬      204 Ì      236 ì
13     45 -      77 M      109 m     141 □      173      205 Í      237 í
14     46 .      78 N      110 n     142 □      174 ®      206 Î      238 î
15     47 /      79 O      111 o     143 □      175 ¯      207 Ï      239 ï
16     48 0      80 P      112 p     144 □      176 °      208 Ð      240 ð
17     49 1      81 Q      113 q     145 □      177 ±      209 Ñ      241 ñ
18     50 2      82 R      114 r     146 □      178 ²      210 Ò      242 ò
19     51 3      83 S      115 s     147 □      179 ³      211 Ó      243 ó
20     52 4      84 T      116 t     148 □      180 ´      212 Ô      244 ô
21     53 5      85 U      117 u     149 □      181 µ      213 Õ      245 õ
22     54 6      86 V      118 v     150 □      182 ¶      214 Ö      246 ö
23     55 7      87 W      119 w     151 □      183 ·      215 ×      247 ÷
24     56 8      88 X      120 x     152 □      184 ¸      216 Ø      248 ø
25     57 9      89 Y      121 y     153 □      185 ¹      217 Ù      249 ù
26     58 :      90 Z      122 z     154 □      186 º      218 Ú      250 ú
27     59 ;      91 [      123 {     155 □      187 »      219 Û      251 û
28     60 <      92 \      124 |     156 □      188 ¼      220 Ü      252 ü
29     61 =      93 ]      125 }     157 □      189 ½      221 Ý      253 ý
30     62 >      94 ^      126 ~     158 □      190 ¾      222 Þ      254 þ
31     63 ?      95 _      127      159 □      191 ¿      223 ß      255 ÿ
```

An algorithm which creates an invisible ink

The encryption can be done with the following algorithm:

For every letter in the text,

- get the Unicode code, we will support only codes 0..255 (8 bits).
Enough for English/Swedish.
 - *just use ord()*
- convert the code to a binary number
 - *we will write function dec2bin()*
- add zeros in front until the code has exactly 8 binary digits
 - *we will write function padzero()*
- replace each 0 with a space, each 1 with a tab
 - *this is a simple loop*

Objects and methods

Object Oriented Programing

Object oriented programing is an abstraction one level above functions.

C++ = C + object oriented programming

Java derives from C++ and tries to be object oriented “all the way” but doesn’t quite achieve it. On the other hand, this also makes some Java programs unnecessarily complicated compared to Python and C.

Python, like C++, supports both object oriented and imperative programming. When it comes to objects, however, it is better than Java.

So far we managed without objects, but we need some of the concepts when we use lists and strings. More about object comes later (Chapter 10 in the book).

An object typically has many methods

```
>>> "hello".upper()  
'HELLO'
```

```
>>> "hello".capitalize()  
'Hello'
```

```
>>> "hello".count("l")  
2
```

```
>>> "hello".join(["A","B","C"])  
'AhelloBhelloC'
```

```
>>> dir(str)  
['__add__', '__class__', '...', 'count',  
'title', 'translate', 'upper', 'zfill']
```

Strings have for example:

Methods can have arguments.

This shows you the list of all methods for a string

Strings of Characters

Indexing and slicing

```
>>> "hello"[0]  
'h'
```

Indexing, the first character is 0

```
>>> "hello"[1:3]  
'el'
```

Slicing: with colon (:), analogous to range()

```
>>> "hello"[:3]  
'hel'
```

From the beginning until 3-1

```
>>> "hello"[3:]  
'lo'
```

From index 3 to the end

```
>>> "hello"[:]  
'hello'
```

The whole string

Indexing and slicing with negative index

```
>>> "hello"[-1]  
'o'
```

Negative indexes are counted from the end.

```
>>> "hello"[-2]  
'l'
```

```
>>> "hello"[:-2]  
'hel'
```

From the beginning without the last two characters.

```
>>> "hello"[-2:]  
'lo'
```

The last two characters.

Index out of range

```
>>> "hello"[10]  
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
IndexError: string index out of range
```

If the index is bigger or equal to the length then we get an error.

Indexing and slicing: step & default values

```
>>> "avocado"[1:6]  
'vocad'
```

Start (inclusive) and stop (exclusive)

```
>>> "avocado"[1:6:2]  
'vcd'
```

Start, stop, **step**: everything between start and stop, with given step.

[start:stop:step]

default: 0 default: end of word default: 1

```
>>> "avocado"[0:3]  
'avo'
```

```
>>> "avocado"[:3]  
'avo'
```

```
>>> "avocado"[3:7]  
'cado'
```

```
>>> "avocado"[3:]  
'cado'
```

```
>>> "avocado"[2:5:1]  
'oca'
```

```
>>> "avocado"[2:5:]  
'oca'
```

function	meaning
<code>s.capitalize()</code>	Copy of <code>s</code> with only the first character capitalized.
<code>s.center(width)</code>	Copy of <code>s</code> centered in a field of given width.
<code>s.count(sub)</code>	Count the number of occurrences of <code>sub</code> in <code>s</code> .
<code>s.find(sub)</code>	Find the first position where <code>sub</code> occurs in <code>s</code> .
<code>s.join(list)</code>	Concatenate <code>list</code> into a string, using <code>s</code> as separator.
<code>s.ljust(width)</code>	Like <code>center</code> , but <code>s</code> is left-justified.
<code>s.lower()</code>	Copy of <code>s</code> in all lowercase characters.
<code>s.lstrip()</code>	Copy of <code>s</code> with leading white space removed.
<code>s.replace(oldsub,newsub)</code>	Replace all occurrences of <code>oldsub</code> in <code>s</code> with <code>newsub</code> .
<code>s.rfind(sub)</code>	Like <code>find</code> , but returns the rightmost position.
<code>s.rjust(width)</code>	Like <code>center</code> , but <code>s</code> is right-justified.
<code>s.rstrip()</code>	Copy of <code>s</code> with trailing white space removed.
<code>s.split()</code>	Split <code>s</code> into a list of substrings (see text).
<code>s.title()</code>	Copy of <code>s</code> with first character of each word capitalized.
<code>s.upper()</code>	Copy of <code>s</code> with all characters converted to uppercase.

Table 5.2: Some string methods

Methods vs functions

```
>>> upper("hello")
NameError: name 'upper' is not defined
```

```
>>> "hello".upper()
'HELLO'
```

```
>>> "hello".len()
AttributeError: 'str' object has no
attribute 'len'
```

```
>>> len("hello")
5
```

```
>>> str.upper("hello")
'HELLO'
```

upper() converts all letters to capital letters.
But it is not available as a function.

Instead it is available as method a in the type str.
To call a method, another syntax is used:

<argument>.<method>()

In contrast **len()** is an ordinary function*

Methods can also be used as normal functions if
the type name is used as a prefix.

* There is no technical reason, just - HCI (Human Computer Interaction), according to Guido van Rossum
<https://stackoverflow.com/questions/237128/why-does-python-code-use-len-function-instead-of-a-length-method>

Formatting of strings

```
>>> print("Sweden","\t",450295)
Sweden      450295
```

```
>>> print("Sweden".ljust(16),str(450295).rjust(10))
Sweden              450295
```

```
>>> print("{0:<16}{1:>10}".format("Sweden",450295))
Sweden              450295
```

```
>>> 10343403/450295
22.970281704216124
```

```
>>> print("with 3 digits: {0:0.3}".format(10343403/450295))
with 3 digits: 23.0
```

The easiest way by using tabs

Full control over the size of the columns: **ljust()** och **rjust()**

The powerful **format()** method

format() can even handle formatting of numbers, for example significant digits

Named arguments to print

```
>>> uni, name = "chalmers", "inari"
```

```
>>> print(name, "@", uni, ".se")
```

```
inari @ chalmers .se
```

```
>>> print(name, "@", uni, ".se", sep="")
```

```
inari@chalmers.se
```

By default, print separates its arguments with space.

Use `sep=str` to override.

```
>>> for i in range(3):
```

```
...     print(i)
```

```
0
```

```
1
```

```
2
```

By default, print ends with a newline.

```
>>> for i in range(3):
```

```
...     print(i, end=" ")
```

```
0 1 2
```

Use `end=str` to override.

Classes of characters

Language independent methods for finding the class of a character. `s.x()` is True when every character in `s` is of the `x` class.

<code>s.isdigit()</code>	True iff every character in <code>s</code> is a digit.
<code>s.isalpha()</code>	True iff every character in <code>s</code> is a letter.
<code>s.isalnum()</code>	True iff every character in <code>s</code> is a letter or a digit.
<code>s.islower()</code>	True iff every character in <code>s</code> is a lower-case letter.
<code>s.isupper()</code>	True iff every character in <code>s</code> is a upper-case letter.
<code>s.isspace()</code>	True iff every character in <code>s</code> is a white space. Includes ' ', '\n', '\r', '\t', etc.

Characters are strings

- Quirk: python does not make a formal difference between characters and strings
- "h" is a string.
- "h"[0] == "h"

(This is not true in most programming languages)

String processing

Example: split text into words

Split text into words separated by spaces and print each word:

```
def words(s):  
    start = 0  
    while start < len(s):  
        while start < len(s) and s[start].isspace():  
            start = start+1  
  
        end = start  
        while end < len(s) and not s[end].isspace():  
            end = end + 1  
  
        print(s[start:end])  
  
        start = end
```



↑
start

Note: while-loops repeat the execution while the condition remains True.

Example: split text into words

Split text into words separated by spaces and print each word:

```
def words(s):  
    start = 0  
    while start < len(s):  
        while start < len(s) and s[start].isspace():  
            start = start+1  
  
        end = start  
        while end < len(s) and not s[end].isspace():  
            end = end + 1  
  
        print(s[start:end])  
  
        start = end
```

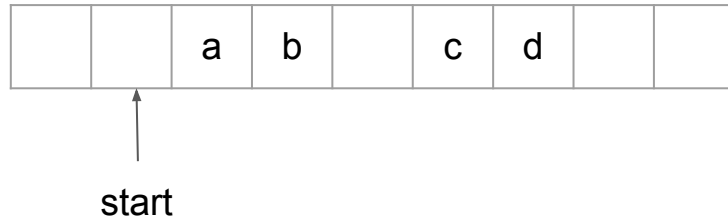


↑
start

Example: split text into words

Split text into words separated by spaces and print each word:

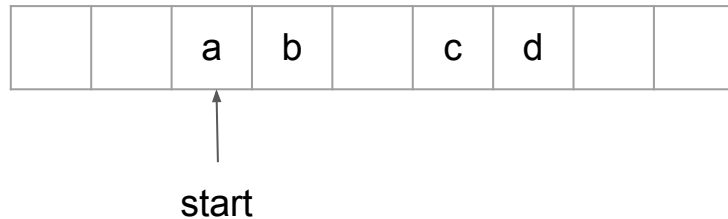
```
def words(s):  
    start = 0  
    while start < len(s):  
        while start < len(s) and s[start].isspace():  
            start = start+1  
  
        end = start  
        while end < len(s) and not s[end].isspace():  
            end = end + 1  
  
        print(s[start:end])  
  
        start = end
```



Example: split text into words

Split text into words separated by spaces and print each word:

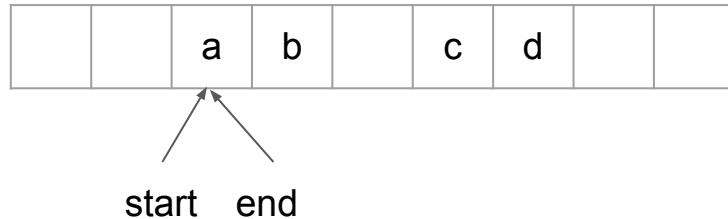
```
def words(s):  
    start = 0  
    while start < len(s):  
        while start < len(s) and s[start].isspace():  
            start = start+1  
  
        end = start  
        while end < len(s) and not s[end].isspace():  
            end = end + 1  
  
        print(s[start:end])  
  
        start = end
```



Example: split text into words

Split text into words separated by spaces and print each word:

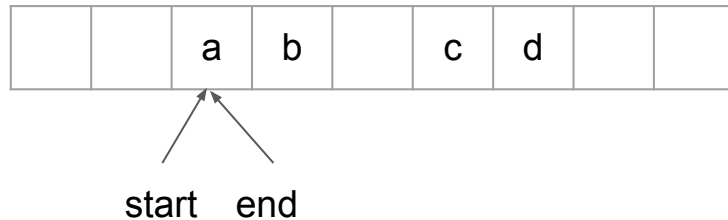
```
def words(s):  
    start = 0  
    while start < len(s):  
        while start < len(s) and s[start].isspace():  
            start = start+1  
  
        end = start  
        while end < len(s) and not s[end].isspace():  
            end = end + 1  
  
        print(s[start:end])  
  
        start = end
```



Example: split text into words

Split text into words separated by spaces and print each word:

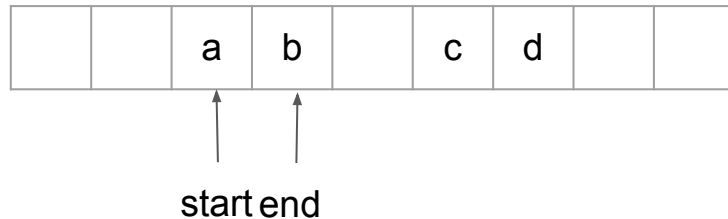
```
def words(s):  
    start = 0  
    while start < len(s):  
        while start < len(s) and s[start].isspace():  
            start = start+1  
  
        end = start  
        while end < len(s) and not s[end].isspace():  
            end = end + 1  
  
        print(s[start:end])  
  
        start = end
```



Example: split text into words

Split text into words separated by spaces and print each word:

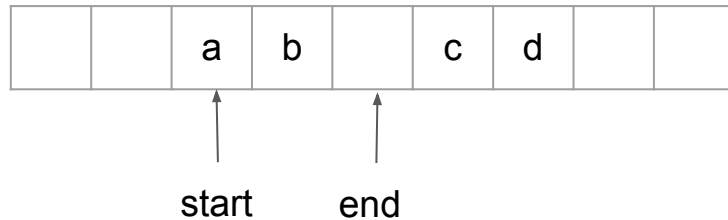
```
def words(s):  
    start = 0  
    while start < len(s):  
        while start < len(s) and s[start].isspace():  
            start = start+1  
  
        end = start  
        while end < len(s) and not s[end].isspace():  
            end = end + 1  
  
        print(s[start:end])  
  
        start = end
```



Example: split text into words

Split text into words separated by spaces and print each word:

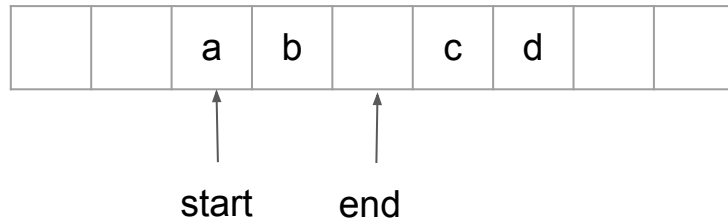
```
def words(s):  
    start = 0  
    while start < len(s):  
        while start < len(s) and s[start].isspace():  
            start = start+1  
  
        end = start  
        while end < len(s) and not s[end].isspace():  
            end = end + 1  
  
        print(s[start:end])  
  
        start = end
```



Example: split text into words

Split text into words separated by spaces and print each word:

```
def words(s):  
    start = 0  
    while start < len(s):  
        while start < len(s) and s[start].isspace():  
            start = start+1  
  
        end = start  
        while end < len(s) and not s[end].isspace():  
            end = end + 1  
  
        print(s[start:end])  
  
        start = end
```



Prints `s[start:end] == "ab"`

Notice the order!

```
while s[start].isspace() and start < len(s):  
    start = start+1
```

An error will happen here

is not the same as:

```
while start < len(s) and s[start].isspace():  
    start = start+1
```

Lists

What is a list?

- We have already used lists informally
- A list is a sequence of values separated by comma and surrounded with square brackets:

```
>>> xs = [3,5,8]      # list of numbers
>>> print(xs)
[3,5,8]
>>> for x in xs:
...     print(x)
3
5
8
```

```
>>> for w in ["first","second","last"]:
...     print(w)
first
second
last
```

Lists and strings

operator	meaning
+	concatenation
*	repetition
<string>[]	indexing
<string>[:]	slicing
len(<string>)	length
for <var> in <string>	iteration through characters

Table 5.1: Python string operations

Zelle, p. 133

operator	meaning
<seq> + <seq>	concatenation
<seq> * <int-expr>	repetition
<seq>[]	indexing
len(<seq>)	length
<seq>[:]	slicing
for <var> in <seq>:	iteration
<expr> in <seq>	membership check (Returns a Boolean)

Python list operations

Zelle, p. 367

All of these work for both strings and lists! (even <exp> in <string>)

You can think about strings as lists of characters but it is not quite true.

Methods for lists as objects

method	meaning
<code><list>.append(x)</code>	Adds element x to end of list.
<code><list>.sort()</code>	Sorts (orders) the list.
<code><list>.reverse()</code>	Reverses the list.
<code><list>.index(x)</code>	Returns index of first occurrence of x .
<code><list>.insert(i,x)</code>	Inserts x into list at index i .
<code><list>.count(x)</code>	Returns the number of occurrences of x in list.
<code><list>.remove(x)</code>	Deletes the first occurrence of x in list.
<code><list>.pop(i)</code>	Deletes the i th element of the list and returns its value.

Some similarities between strings and lists

```
>>> s = '1666'
>>> xs = [1,6,6,6]
```

```
>>> s.count('6')
```

```
3
```

```
>>> xs.count(6)
```

```
3
```

```
>>> 'A' + s
```

```
'A1666'
```

```
>>> ['A'] + xs
```

```
['A',1,6,6,6]
```

```
>>> 'A' + xs
```

```
TypeError: can only concatenate str
(not "list") to str
```

count() works in the same way

+ works in the same way

here we have a list concatenation

But note that it must be only:

<list> + <list>

<str> + <str>

and not some combination.

Mutable objects

Lists are mutable: they can be changed

```
>>> xs = [1,2,3,4]
```

We create a list.

```
>>> xs[0] = 0
```

We change the first element.

```
>>> xs  
[0, 2, 3, 4]
```

```
>>> s = '1234'
```

We create a string.

```
>>> s[0] = '0'
```

But strings cannot be changed.

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
TypeError: 'str' object does not  
support item assignment
```

The difference: lists are **mutable**, strings are not.

Lists are mutable: they can be extended

```
>>> xs = [1,2,3,4]
```

We create a list.

```
>>> xs.append(10)
```

We add one more element.

```
>>> xs
```

```
[1, 2, 3, 4, 10]
```

Lists are mutable: values can be removed

```
>>> xs = [1,2,3,4]
```

```
>>> last_item = xs.pop()
```

```
>>> xs  
[1, 2, 3]
```

```
>>> last_item  
4
```

```
>>> xs = [1,2,3,4]
```

```
>>> first_item = xs.pop(0)
```

```
# xs is now [2, 3, 4], first_item is 1
```

We create a list.

The method `.pop()` does two things:

- *removes* an element (last if not specified)
- *returns* it

`.pop(i)` removes and returns the element at index `i`

Lists are mutable: notice even this

```
>>> xs = [1,2,3,4]
```

```
>>> ys = xs
```

```
>>> xs[0] = 0
```

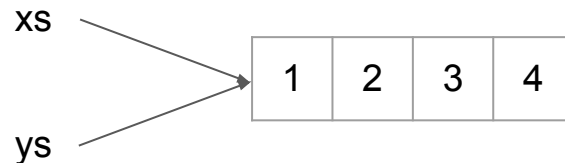
```
>>> ys.append(10)
```

```
>>> xs
```

```
[0, 2, 3, 4, 10]
```

```
>>> ys
```

```
[0, 2, 3, 4, 10]
```



Variables **`xs`** and **`ys`** are just two *references* to the same object. Any change to the object is visible from either variable.

Access or change several elements

```
>>> xs = [1,2,3,4]

>>> for i in range(len(xs)): xs[i] = xs[i] + 10
...
>>> xs
[11, 12, 13, 14]

>>> xs = [1,2,3,4]

>>> s = 0
>>> for x in xs: s = s + x
...
>>> xs
[1, 2, 3, 4]
>>> s
10
```

We create a list.

Change the value in every position in a loop

The whole list is changed.

But now try this again.

Unlike the previous loop, this one goes over the elements, but now we cannot change them!

xs is still the same!

But now we computed the sum.

Lists are changed in-place

```
>>> xs = [1,6,6,6]
```

```
>>> xs.reverse()
```

```
>>> type(xs.reverse())  
<class 'NoneType'>
```

```
>>> xs  
[6,6,6,1]
```

```
>>> "hello".reverse()  
AttributeError: 'str' object has no  
attribute 'reverse'
```

reverse() reverses the list

But it is done in-place! The method doesn't return a value, but just like **print()** it causes side effects

You can now see that the list has been changed.

Strings don't have a **reverse()** method. But, how can we reverse a string?

Summary

Mutable objects (for example lists) can be changed in place, without creating new objects.

Immutable objects (strings and integers) cannot be changed in place: new objects must be created for every new value.

The value of a variable can always be changed to point to a new object.

**There are some subtle differences here but we will come back to that later.
We should first cover the basics.**

Conversion between strings and lists

```
>>> cs = list("hello")

>>> cs
['h', 'e', 'l', 'l', 'o']

>>> cs.reverse()

>>> "".join(cs)
'olleh'

>>> str(cs)
"['o', 'l', 'l', 'e', 'h']"
```

list() converts the string to a list of characters

(A character in Python is just a string of length 1)

The list can be reverted!

<str>.join(<list>) concatenates the strings in **<list>** and puts **<str>** in between. The result is the string reversed.

Note: **str()** doesn't give the same conversion: it just shows an expression for the list in double quotes.

Note: Python automatically chooses to use double quotes if the string itself contains single quotes.

Tuples

(immutable lists)

Tuples vs Lists

List:

- creation: `xs = [1,2,3,4]`
- indexing: `xs[1] -> 2`
- slicing: `xs[1:3] -> [2,3]`
- update: `xs[1] = 0`

Tuple:

- creation: `xs = (1,2,3,4)`
- indexing: `xs[1] -> 2`
- slicing: `xs[1:3] -> (2,3)`
- update: **not possible!**

- All immutable operations for lists are available for tuples as well.
- None of the mutable operations for lists (append, reverse, etc) works for tuples.
- Alternative syntax for creating tuples, just use comma: `1,2,3,4 == (1,2,3,4)`
 - except in argument lists: `f(1,2,3,4) != f((1,2,3,4))`

Pattern matching (list)

```
>>> xs = [3,4,5]
```

```
>>> [a,b,c] = xs
```

```
>>> print(a)
```

```
3
```

```
>>> print(b)
```

```
4
```

```
>>> print(c)
```

```
5
```

The same as:

```
a = xs[0]
```

```
b = xs[1]
```

```
c = xs[2]
```

Pattern matching (tuple)

```
>>> xs = (3,4,5)
```

```
>>> (a,b,c) = xs
```

```
>>> print(a)
```

```
3
```

```
>>> print(b)
```

```
4
```

```
>>> print(c)
```

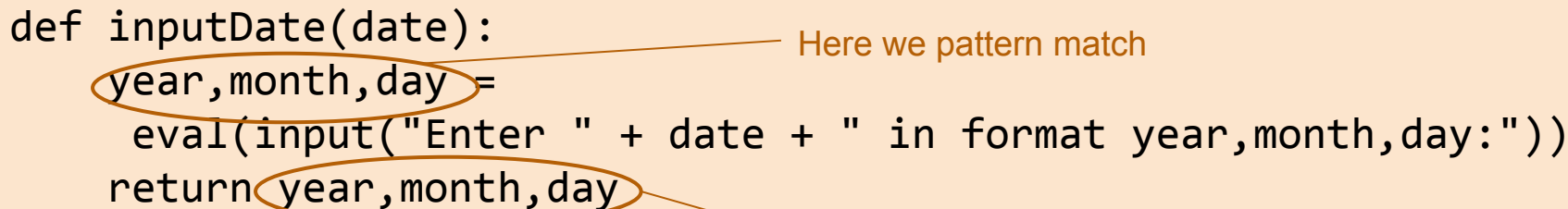
```
5
```

Pattern matching works equally well with tuples.

It is actually a lot more often used for tuples than for lists.

Recall from lecture 2

```
def inputDate(date):  
    year, month, day =  
        eval(input("Enter " + date + " in format year,month,day:"))  
    return year, month, day
```



The diagram illustrates the execution of the `inputDate` function. It shows a function definition with a parameter `date`. Inside the function, a tuple `(year, month, day)` is assigned the result of `eval(input(...))`. The tuple `(year, month, day)` is circled in orange, and a line points from this circle to the text "Then we created a new tuple". Another line points from the text "Here we pattern match" to the assignment `year, month, day =`.

Here we pattern match

Then we created a new tuple

- Here we had actually asked the user to type a tuple of numbers, and `eval` converts it to an actual value.

Back to the invisible ink

An algorithm which creates an invisible ink

The encryption can be done with the following algorithm:

For every letter in the text,

- get the Unicode code, we will support only codes 0..255 (8 bits).
Enough for English/Swedish.
 - *just use ord()*
- convert the code to a binary number
 - *we will write function dec2bin()*
- add zeros in front until the code has exactly 8 binary digits
 - *we will write function padzero()*
- replace each 0 with a space, each 1 with a tab
 - *this is a simple loop*

base 2 numeral system

1	0	1	1
----------	----------	----------	----------

$$1*2^3 + 0*2^2 + 1*2^1 + 1*2^0$$

$$1*8 + 0*4 + 1*2 + 1*1$$

$$8 + 0 + 2 + 1$$

$$= 11$$

Every position corresponds to a power of 2.

1	7	9	3
----------	----------	----------	----------

$$1*10^3 + 7*10^2 + 9*10^1 + 3*10^0$$

$$1*1000 + 7*100 + 9*10 + 3*1$$

$$1000 + 700 + 90 + 3$$

$$= 1793$$

Compare with decimal numbers:
Every position corresponds to a power of 10.

Algorithm for conversion from binary to decimal

```
def bin2dec(bin):  
    dec = 0  
    for i in range(len(bin)):  
        dec = dec + (2**(len(bin)-i-1)) * int(bin[i])  
    return dec
```

Example: bin = "10111"

len(bin) → 5

dec = 0

i = 0:

dec = 0 + 2**4 * 1 → 16

i = 1:

dec = 16 + 2**3 * 0 → 16

i = 2:

dec = 16 + 2**2 * 1 → 20

i = 3:

dec = 20 + 2**1 * 1 → 22

i = 4:

dec = 22 + 2**0 * 1 → 23

return 23

Algorithm for conversion from decimal to binary

```
def dec2bin(dec):  
    bin = ""  
    while dec > 0:  
        bit = str(dec % 2)  
        bin = bit + bin  
        dec = dec // 2  
    return bin
```

Example: dec = 23

bin = ""

bit = "1"; bin = "1"

dec = 11

bit = "1"; bin = "11"

dec = 5

bit = "1"; bin = "111"

dec = 2

bit = "0"; bin = "0111"

dec = 1

bit = "1"; bin = "10111"

return "10111"

Padding and txt2bin()

```
def padzero(bin,n):  
    need = n - len(bin)  
    bin = need * '0' + bin  
    return bin  
  
def txt2bin(txt):  
    bin = []  
    for c in txt:  
        bin.append(padzero(dec2bin(ord(c)),8))  
    return "".join(bin)
```

We add zeros until we get length n

- **need** is the number of zeros needed
- **bin** is expanded with zeros

We glue together all 8-bit strings to a long string.

Here we need **ord()** and **dec2bin()**

Quiz: *why do we need the padding actually?*

The rest of txt2invisible()

```
def bin2invisible(bin):  
  
    inv = []  
    for b in bin:  
        if b=='0':  
            inv.append(' ')  
        else:  
            inv.append('\t')  
    return "".join(inv)  
  
def txt2invisible(txt):  
    return bin2invisible(txt2bin(txt))
```

We convert every 0 to ' ', every 1 to '\t'.

Note that inv is a list, not a string

xs.append(c) adds the element c to the list xs.

s.join(xs) creates a string by gluing together all the elements in xs with s in between. Here s is the empty string "".

At the end we combine the functions.

An algorithm for reading the invisible ink

You need the opposite steps. To be done in Exercise 1:

Take the string in the invisible text and:

- replace every space with 0, every tab with 1
 - *a simple for loop*
- divide the bits into groups of 8
 - *now we see why we needed the padding*
- convert the "bytes" to decimal numbers
 - *use bin2dec in a loop which goes over the bytes*
- apply chr() to every number
 - *the result is a list of characters which can be joined to a string*

Variant: read between the lines

*The nuclear codes are in the
black suitcase on the bench.
Password kanelbulle.*

Hej,

Jag undrar om du kommer ihåg vilka övningar

vi måste lämna in och vilken deadline vi har.

Om du inte kommer ihåg kan jag fråga någon

annan och berätta även för dig om du vill.

Sending a file which looks “empty” is too suspicious. A programmer may crack the code quite easily. One variant is to hide the message in "empty" lines between normal lines of text.

Left as extra exercise!

Variant 2: read between the words

*The nuclear codes are in the
black suitcase on the bench.
Password kanelbulle.*

Hej,

Jag undrar om du kommer ihåg vilka övningar vi måste lämna in och vilken deadline vi har. Om du inte kommer ihåg kan jag fråga någon annan och berätta även för dig om du vill. Vi har ju fått så mycket att göra på den här förbaskade kursen att ingen kan klara det alldeles själv. Så det är jätteviktigt att vi alla hjälps åt och gör vad vi

Empty lines are still too visible, it is even better to do the encoding by spaces between the words:

- 0 = one space
- 1 = two (or more) spaces (a tab would be too visible and might be suspicious)

Left as extra exercise!

References

From the book

Lecture:

- Chapter 5
- More about list operations and methods in 11.2.2

Lab 1:

- Strings, lists allow you to start working on Lab 1 already.
- The rest (dictionaries, files) we will discuss in the next lecture.